

EN3160 Assignment 1

Intensity Transformations and Neighborhood Filtering

Name : Ahamed A.M.S

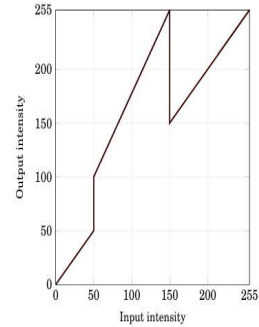
Index No : 230020R

GitHub : <https://github.com/shaahidahamed>

1. Implement the intensity transformation depicted in Fig. 1a on the image shown in Fig. 1b by writing a function `intensity_transform(im, breakpoints)` where the break points are, e.g.,

$$\begin{bmatrix} 0 & 0 \\ 50 & 50 \\ 100 & 150 \\ 150 & 255 \\ 255 & 255 \end{bmatrix}$$

Experiment with different break points to get a visually pleasing output. Show the intensity transformation as a plot and the original and transformed images side-by-side.



(a) Intensity transformation.



(b) Image for intensity transformation.

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

img = cv.imread('./images/emma.jpg', cv.IMREAD_GRAYSCALE)

def intensity_transform(im, breakpoints):
    breakpoints = np.asarray(breakpoints)
    x = breakpoints[:, 0]
    y = breakpoints[:, 1]
    transformed = np.interp(im, x, y)
    return np.clip(transformed, 0, 255).astype(np.uint8)

breakpoints = np.array([
    [0, 0], [50, 50], [50, 100],
    [150, 255], [150, 150], [255, 255]])

result = intensity_transform(img, breakpoints)

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.title('Original Image')
plt.imshow(img, cmap='gray', vmin=0, vmax=255)
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title('Transformed Image')
plt.imshow(result, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.show()

plt.figure(figsize=(6, 5))
plt.plot(breakpoints[:, 0], breakpoints[:, 1])
plt.xlim(0, 260)
plt.ylim(0, 260)
plt.xlabel("Input intensity")
plt.ylabel("Output intensity")
plt.title("Intensity Transformation")
plt.grid()
plt.show()
```

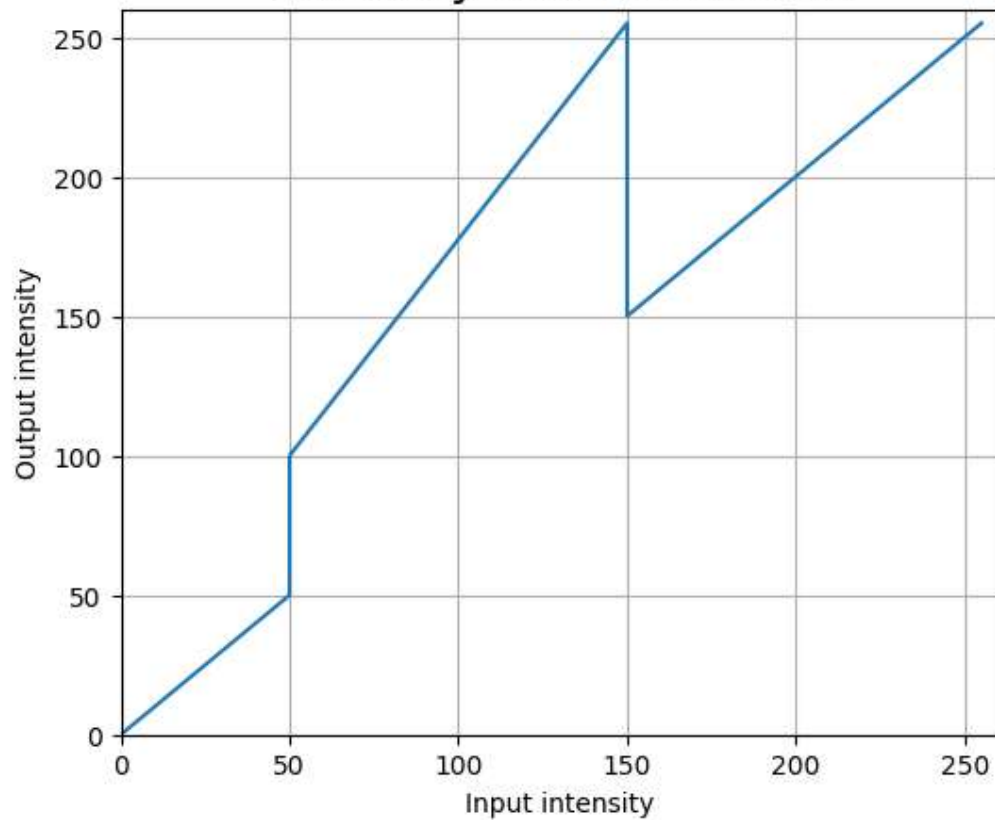
Original Image



Transformed Image



Intensity Transformation



Discussion: The piecewise intensity transformation mainly enhances the mid-range pixel values between 50 and 150, increasing their brightness and contrast. Pixel values below 50 and above 150 remain mostly unchanged. As a result, mid-tone facial regions become more prominent, while darker regions such as the hair and brighter regions show only minor changes.

2. Apply a similar operation as above (question 1) to accentuate

- (a) white matter
- (b) gray matter

in the brain proton density image shown in Fig. 2. Show the intensity transformations as a plots.

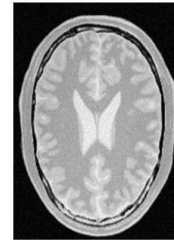


Figure 2: A brain proton density slice.

```
brain = cv.imread('./images/brain_proton_density_slice.png', cv.IMREAD_GRAYSCALE)

white_bp = np.array([
    [0, 0], [150, 15], [150, 245],
    [175, 247], [175, 17.5], [255, 25.5]])

white_matter = intensity_transform(brain, white_bp)

gray_bp = np.array([
    [0, 0], [185, 18.5], [185, 248.5],
    [210, 251], [210, 21], [255, 25.5]])

gray_matter = intensity_transform(brain, gray_bp)

plt.figure(figsize=(15, 4))

plt.subplot(1, 3, 1)
plt.imshow(brain, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(white_matter, cmap="gray")
plt.title("White Matter Accentuated")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(gray_matter, cmap="gray")
plt.title("Gray Matter Accentuated")
plt.axis("off")

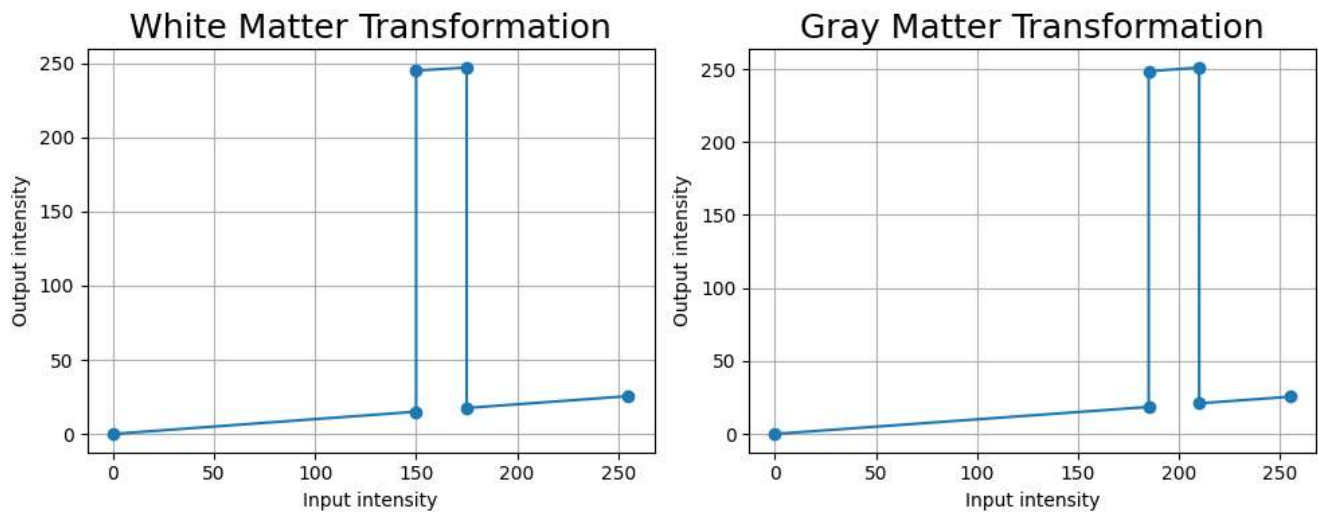
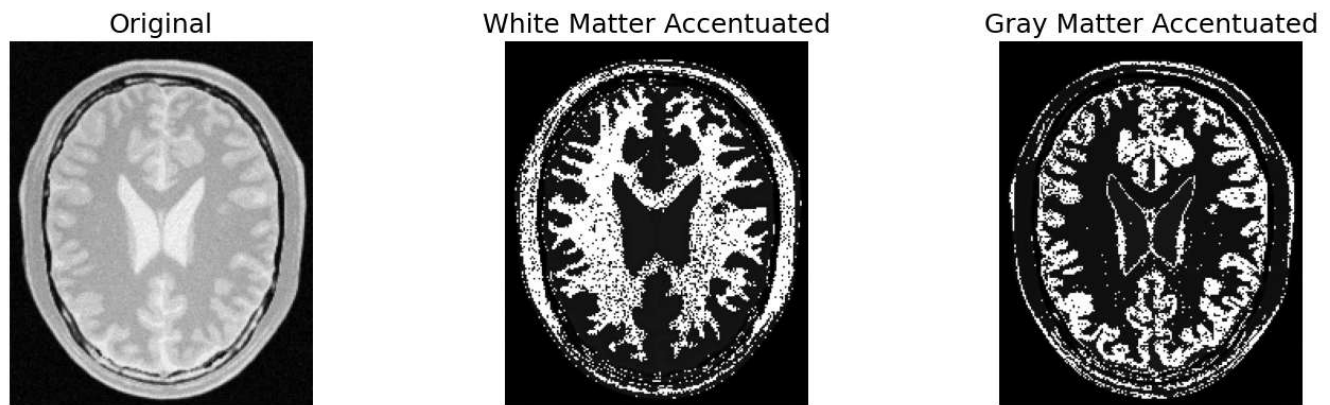
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(white_bp[:,0], white_bp[:,1], marker='o')
plt.xlabel("Input intensity")
plt.ylabel("Output intensity")
plt.title("White Matter Transformation")
plt.grid()

plt.subplot(1, 2, 2)
plt.plot(gray_bp[:,0], gray_bp[:,1], marker='o')
plt.xlabel("Input intensity")
plt.ylabel("Output intensity")
plt.title("Gray Matter Transformation")
plt.grid()

plt.tight_layout()
plt.show()
```



Discussion: White matter and gray matter were emphasized by selecting specific intensity ranges in the brain image. The ranges 150-175 and 185-210 were mapped to higher output intensities, making the corresponding tissue regions more visible. This shows how intensity slicing can be used to highlight structures with specific gray-level values.

3. Consider the image shown in Fig. 3¹.

- Apply gamma correction to the L plane in the $L^*a^*b^*$ color space and state the γ value.
- Show the histograms of the original and corrected images.



Figure 3: Image for gamma correction.

```
img = cv.imread('./images/highlights_and_shadows.jpg')
lab = cv.cvtColor(img, cv.COLOR_BGR2LAB)
L, a, b = cv.split(lab)

gamma = 0.6

t = np.array([(i / 255) ** gamma * 255 for i in range(256)]).astype(np.uint8)

g = cv.LUT(L, t)
corrected = cv.merge([g, a, b])
corrected_rgb = cv.cvtColor(corrected, cv.COLOR_LAB2RGB)

plt.figure(figsize=(5, 4))
plt.plot(t)
plt.title(f'Gamma Transformation ( $\gamma = \{gamma\}$ )')
plt.xlabel('Input Intensity')
plt.ylabel('Output Intensity')
plt.xlim(0, 255)
plt.ylim(0, 255)
```

```

plt.grid()
plt.show()

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB))
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(corrected_rgb)
plt.title('Gamma Corrected Image')
plt.axis('off')

plt.tight_layout()
plt.show()

hist1 = cv.calcHist([L], [0], None, [256], [0, 256])
hist2 = cv.calcHist([g], [0], None, [256], [0, 256])

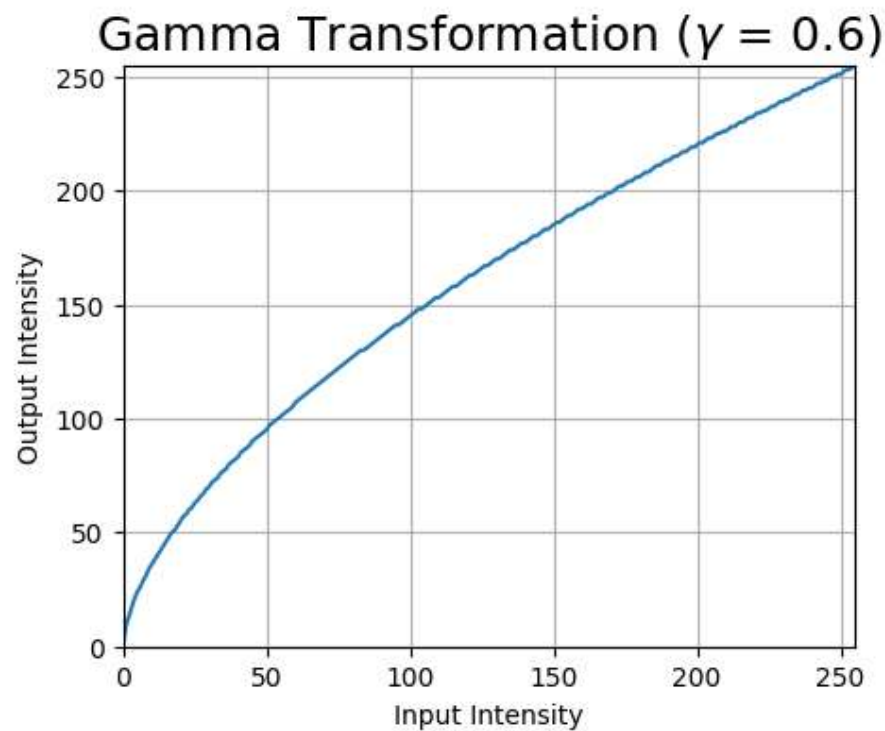
plt.figure(figsize=(8, 5))

plt.plot(hist1, label='Original')
plt.plot(hist2, label='Gamma Corrected')

plt.title('Intensity Histogram')
plt.xlabel('Intensity')
plt.ylabel('Count')
plt.xlim(0, 256)
plt.legend()

plt.show()

```



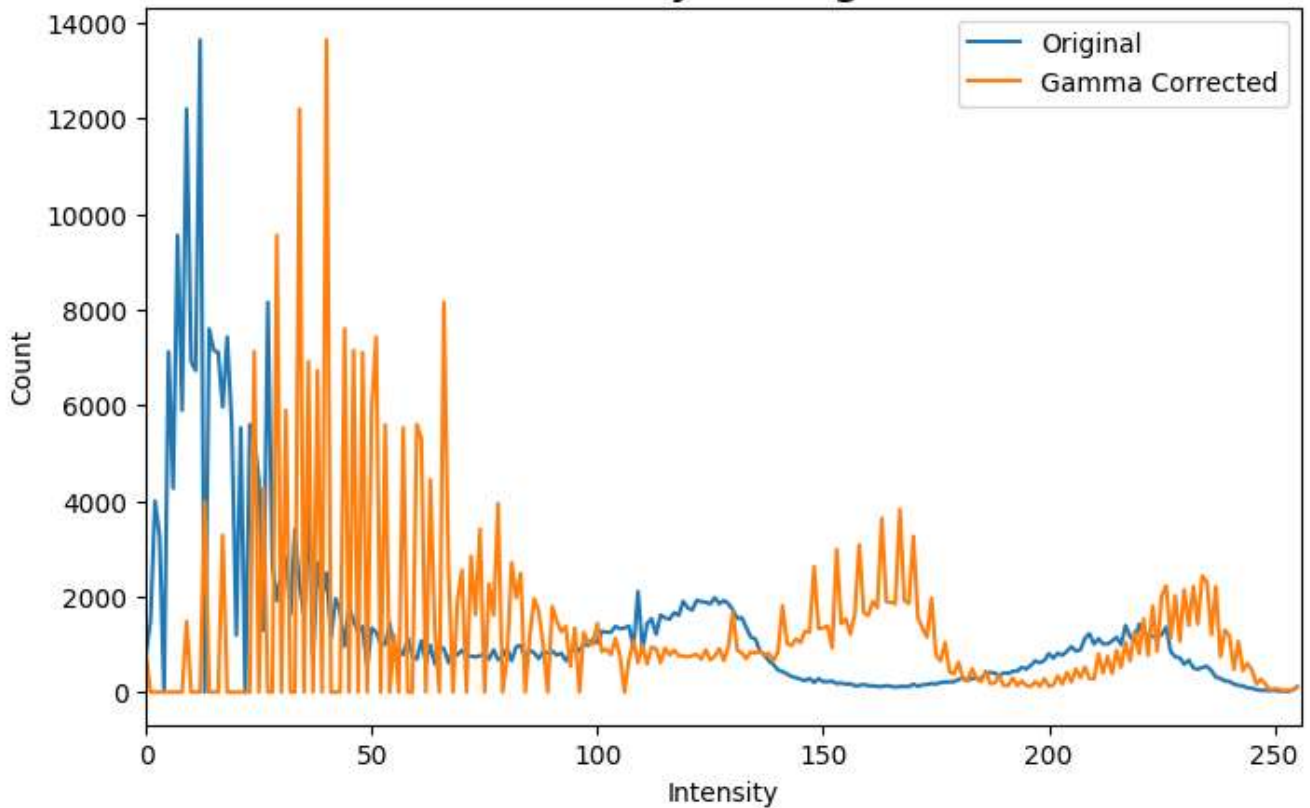
Original Image



Gamma Corrected Image



Intensity Histogram



Discussion: Gamma correction was applied only to the L plane of the L*a*b image. A gamma value of 0.6 increased the brightness of darker regions and revealed more detail in the shadows. Since only the L plane was modified, the color information in the a and b planes remained unchanged, preserving the overall color appearance.

4. Increasing the vibrance of a photograph is probably achieved by applying an intensity transformation such as

$$f(x) = \min \left(x + a \times 128 e^{-\frac{(x-128)^2}{2\sigma^2}}, 255 \right),$$

to the saturation plane, where x is the input intensity, $a \in [0, 1]$ and $\sigma = 70$.

- Split the image shown in Fig. 4 into hue, saturation, and value planes.
- Apply the aforementioned intensity transformation to the saturation plane.
- Adjust a to get a visually pleasing output. Report the value of a .
- Recombine the three planes.
- Display the original image, vibrance-enhanced image, and the intensity transformation.



Figure 4: Image for enhancing the vibrance.

```

img = cv.imread('./images/spider.png', cv.IMREAD_COLOR)
hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)
H, S, V = cv.split(hsv)

plt.figure(figsize=(15, 4))

plt.subplot(1, 3, 1), plt.imshow(H, cmap="gray"), plt.title("Hue"), plt.axis("off")
plt.subplot(1, 3, 2), plt.imshow(S, cmap="gray"), plt.title("Saturation"), plt.axis("off")
plt.subplot(1, 3, 3), plt.imshow(V, cmap="gray"), plt.title("Value"), plt.axis("off")

plt.tight_layout()
plt.show()

def vibrance_transform(S, a, sigma=70):
    x = S.astype(np.float32)
    result = x + a * 128 * np.exp(-((x - 128) ** 2) / (2 * sigma ** 2))
    result = np.minimum(result, 255)
    return result.astype(np.uint8)

x = np.arange(256)
sigma = 70
a_values = [0.2, 0.4, 0.6, 0.8, 1.0]

plt.figure(figsize=(8, 6))

for a in a_values:
    y = np.minimum(x + a * 128 * np.exp(-((x - 128) ** 2) / (2 * sigma ** 2)), 255)
    plt.plot(x, y, label=f"a = {a}")

plt.plot(x, x, '--', label="Identity")

plt.xlabel("Input Saturation")
plt.ylabel("Output Saturation")
plt.title("Vibrance Transformation for Different a Values")

plt.xlim(0, 255)
plt.ylim(0, 270)

plt.legend()
plt.grid()

plt.show()

a_value = 0.4
S_new = vibrance_transform(S, a_value)
hsv_new = cv.merge([H, S_new, V])

vibrant = cv.cvtColor(hsv_new, cv.COLOR_HSV2RGB)
original = cv.cvtColor(img, cv.COLOR_BGR2RGB)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(original)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(vibrant)
plt.title(f"Vibrance Enhanced Image, a = {a_value}")
plt.axis("off")

plt.tight_layout()
plt.show()

```

Hue



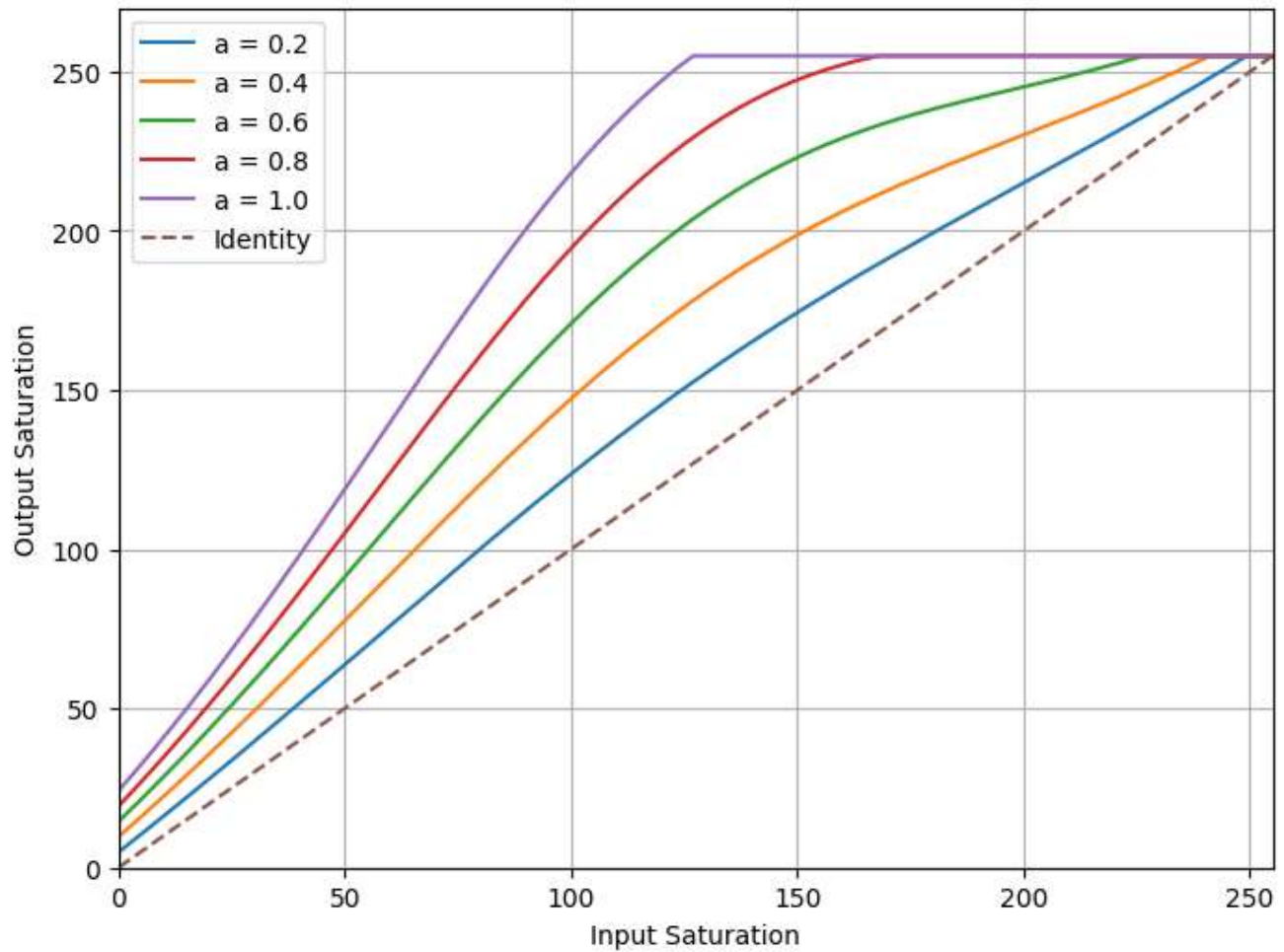
Saturation



Value



Vibrance Transformation for Different a Values



Original Image



Vibrance Enhanced Image, a = 0.4



Discussion: The saturation plane was modified using the given vibrance transformation to make the colors more vivid. Several a values were tested, and larger values produced stronger saturation. A value of $a = 0.4$ was selected because it enhanced the colors while keeping the image natural and avoiding excessive saturation.

5. Write a function of your own to carry out histogram equalization on the image shown in Fig. 5. Show the histograms before and after equalization.

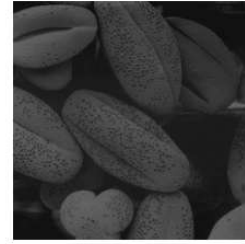


Figure 5: Image for histogram equalization.

```
def histogram_equalization(img):
    hist, _ = np.histogram(img.ravel(), 256, [0, 256])
    cdf = np.cumsum(hist)
    total_pixels = img.shape[0] * img.shape[1]
    transform = np.round(cdf * 255 / total_pixels).astype(np.uint8)
    equalized_img = transform[img]

    return equalized_img

img = cv.imread('./images/shells.tif', cv.IMREAD_GRAYSCALE)
equalized_img = histogram_equalization(img)

before_hist, _ = np.histogram(img.ravel(), 256, [0, 256])
after_hist, _ = np.histogram(equalized_img.ravel(), 256, [0, 256])

plt.figure(figsize=(20, 5))
plt.rc("axes", titlesize=18)

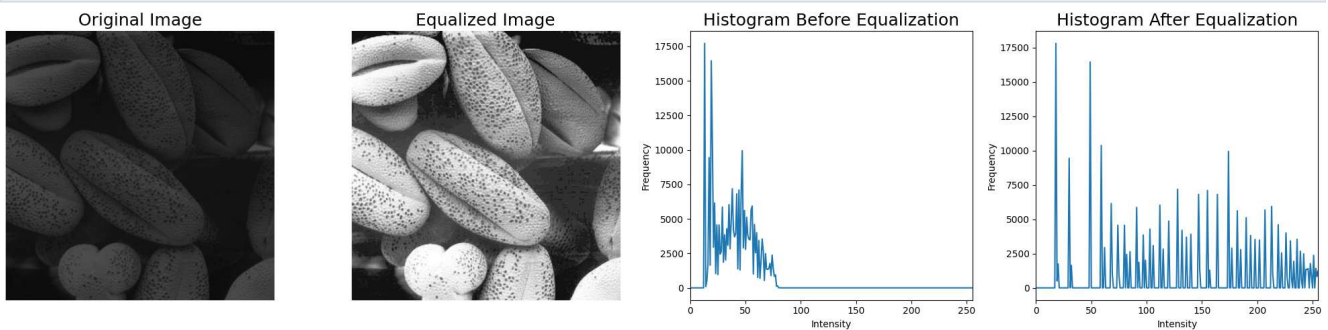
plt.subplot(1, 4, 1)
plt.imshow(img, cmap="gray", vmin=0, vmax=255)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(equalized_img, cmap="gray", vmin=0, vmax=255)
plt.title("Equalized Image")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.plot(before_hist)
plt.title("Histogram Before Equalization")
plt.xlabel("Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 255])

plt.subplot(1, 4, 4)
plt.plot(after_hist)
plt.title("Histogram After Equalization")
plt.xlabel("Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 255])

plt.tight_layout()
plt.show()
```



Discussion: Histogram equalization redistributes the image intensities over a wider range using the cumulative distribution function. This improves the contrast of the image and makes darker details more visible. The resulting histogram is more spread out compared with the original, indicating better use of the available intensity range.

6. In this question, we will apply histogram equalization only to the foreground of an image to produce an image with a histogram equalized foreground.

- Open the image in Fig. 6, split it into hue, saturation, and values and display these planes in grayscale.
- Select the appropriate plane to threshold in extract the foreground mask. A mask is a binary image.
- Now obtain the foreground only using `cv.bitwise_and` and compute the histogram.
- Obtain the cumulative sum of the histogram using `np.cumsum`.
- Use the formulas in slides to histogram-equalize the foreground.
- Extract the background and add with the histogram equalized foreground.

Show the hue, saturation, and value plane, the mask, the original image, and the result with the histogram-equalized foreground.



Figure 6: Image for histogram equalizing the foreground

```
img=cv.imread("./images/jeniffer.jpg",cv.IMREAD_COLOR)
hsv=cv.cvtColor(img,cv.COLOR_BGR2HSV)
H,S,V=cv.split(hsv)
plt.figure(figsize=(15,4))
for i,(plane,title) in enumerate([(H,"Hue Plane"),(S,"Saturation Plane"),(V,"Value Plane")]):
    plt.subplot(1,3,i+1)
    plt.imshow(plane,cmap="gray")
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()

threshold=15
mask=cv.inRange(S,threshold,255)

kernel=cv.getStructuringElement(cv.MORPH_ELLIPSE,(80,80))
mask=cv.morphologyEx(mask,cv.MORPH_CLOSE,kernel)

foreground=cv.bitwise_and(img,img,mask=mask)
original=cv.cvtColor(img,cv.COLOR_BGR2RGB)
foreground_rgb=cv.cvtColor(foreground,cv.COLOR_BGR2RGB)

plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(original)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(mask,cmap="gray")
plt.title("Foreground Mask")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(foreground_rgb)
plt.title("Foreground")
plt.axis("off")

plt.tight_layout()
plt.show()

hist=cv.calcHist([V],[0],mask,[256],[0,256])
cdf=np.cumsum(hist)

plt.figure(figsize=(15,5))
```

```

plt.subplot(1,2,1)
plt.plot(hist)
plt.xlim([0,256])
plt.title("Histogram of Foreground")

plt.subplot(1,2,2)
plt.plot(cdf)
plt.xlim([0,256])
plt.title("Cumulative Sum of Histogram")

plt.tight_layout()
plt.show()

mapping=cdf*255/cdf[-1]
V_equalized=V.copy()
V_equalized[mask>0]=mapping[V[mask>0]]

equalized_hsv=cv.merge([H,S,V_equalized])
equalized_foreground=cv.cvtColor(equalized_hsv,cv.COLOR_HSV2RGB)
mask_3d=np.repeat(mask[:, :,None],3,axis=2)
equalized_foreground=np.bitwise_and(equalized_foreground,mask_3d)
background_mask=cv.bitwise_not(mask)
background=cv.bitwise_and(img,img,mask=background_mask)
background_rgb=cv.cvtColor(background,cv.COLOR_BGR2RGB)
final_image=cv.add(background_rgb,equalized_foreground)

plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(equalized_foreground)
plt.title("Equalized Foreground")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(final_image)
plt.title("Final Result")
plt.axis("off")

plt.tight_layout()
plt.show()

```

Hue Plane



Saturation Plane



Value Plane



Original Image

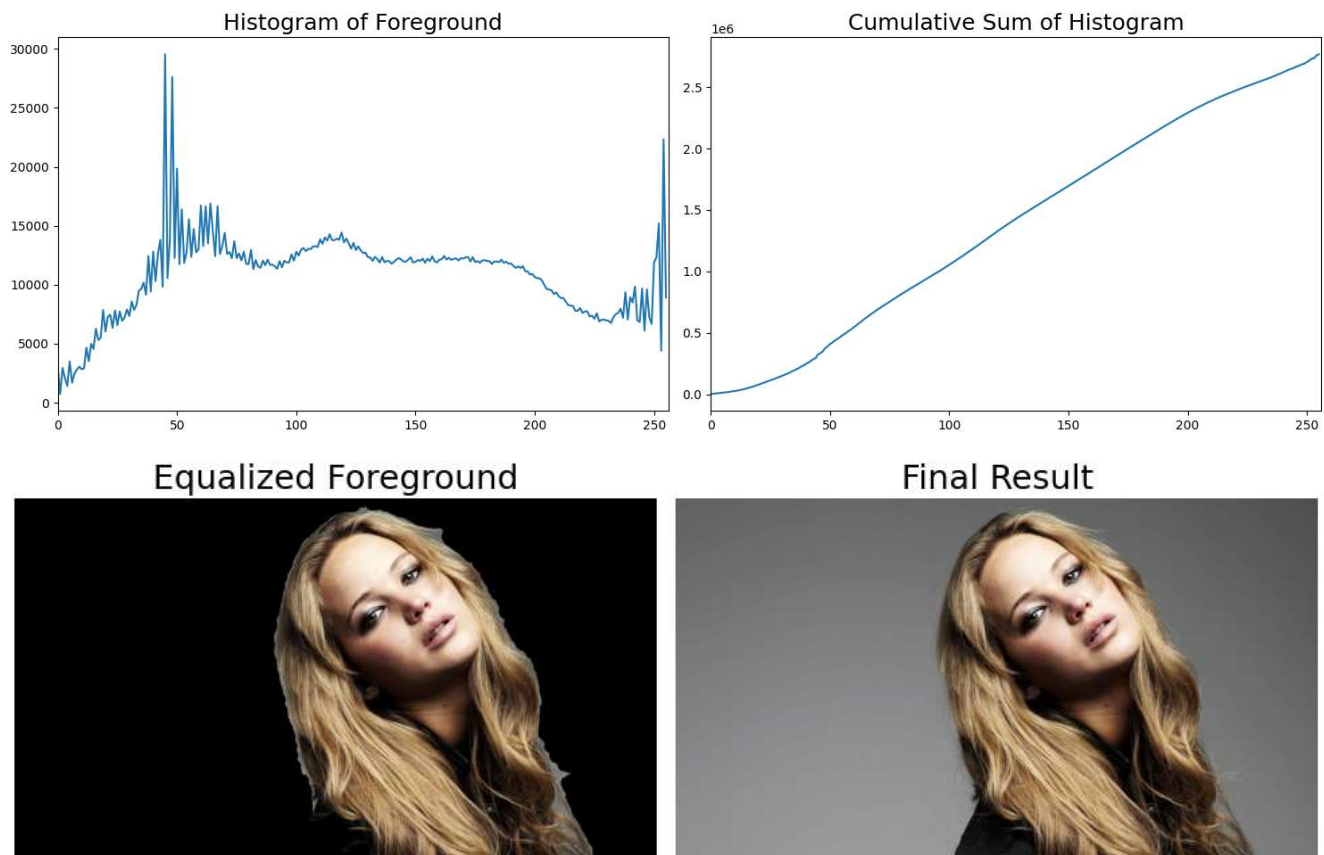


Foreground Mask



Foreground





Discussion: The saturation plane was selected for thresholding because the foreground has higher saturation than most of the background. Morphological operations were used to improve the binary mask. Histogram equalization was then applied only to the foreground value intensities, while the original background was preserved and combined with the enhanced foreground.

7. Filtering with the Sobel operator can compute the gradient. Consider the image shown in Fig. 7

- Using the existing filter2D to Sobel filter the image.
- Write your own code to Sobel filter the image.
- Using the property

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \end{bmatrix},$$

carry out Sobel filtering.

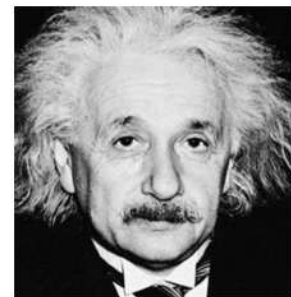


Figure 7: Image for Sobel filtering.

```
img=cv.imread('./images/einstein.png',cv.IMREAD_GRAYSCALE).astype(np.float32)

Sobel_V_kernel=np.array([[1,0,-1],[2,0,-2],[1,0,-1]],dtype=np.float32)
Sobel_H_kernel=np.array([[1,2,1],[0,0,0],[-1,-2,-1]],dtype=np.float32)

# (a) Using filter2D
sobel_v=cv.filter2D(img,cv.CV_32F,Sobel_V_kernel)
sobel_h=cv.filter2D(img,cv.CV_32F,Sobel_H_kernel)
magnitude=np.sqrt(sobel_v**2+sobel_h**2)

# (b) Own Sobel filtering code
def my_filter2d(image,kernel):
    kh,kw=kernel.shape
    padded=np.pad(image,((kh//2,kh//2),(kw//2,kw//2)),mode='reflect')
    output=np.zeros_like(image,dtype=np.float32)

    for y in range(image.shape[0]):
        for x in range(image.shape[1]):
            region=padded[y:y+kh,x:x+kw]
```

```

        output[y,x]=np.sum(region*kernel)
    return output

sobel_v_my=my_filter2d(img,Sobel_V_kernel)
sobel_h_my=my_filter2d(img,Sobel_H_kernel)
magnitude_my=np.sqrt(sobel_v_my**2+sobel_h_my**2)

# (c) Using separable property
smooth=np.array([1,2,1],dtype=np.float32)
derivative=np.array([1,0,-1],dtype=np.float32)

sobel_v_sep=cv.sepFilter2D(img,cv.CV_32F,derivative,smooth)
sobel_h_sep=cv.sepFilter2D(img,cv.CV_32F,smooth,derivative)
magnitude_sep=np.sqrt(sobel_v_sep**2+sobel_h_sep**2)

images=[
    sobel_v,sobel_h,magnitude,
    sobel_v_my,sobel_h_my,magnitude_my,
    sobel_v_sep,sobel_h_sep,magnitude_sep
]

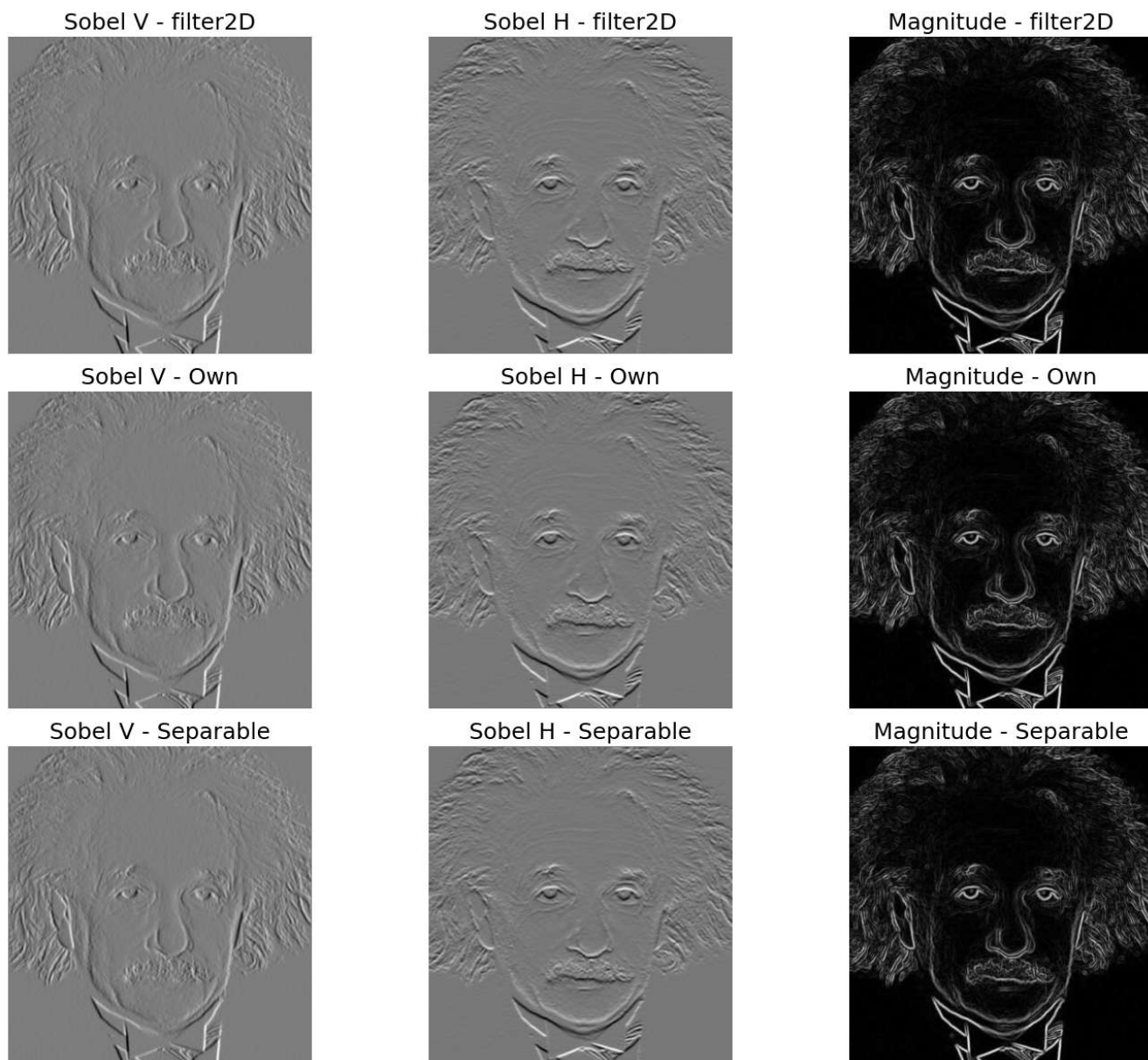
titles=[
    'Sobel V - filter2D','Sobel H - filter2D','Magnitude - filter2D',
    'Sobel V - Own','Sobel H - Own','Magnitude - Own',
    'Sobel V - Separable','Sobel H - Separable','Magnitude - Separable'
]

plt.figure(figsize=(15,12))

for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(images[i],cmap='gray')
    plt.title(titles[i])
    plt.axis('off')

plt.tight_layout()
plt.show()

```



Discussion: The Sobel operator was used to calculate horizontal and vertical image gradients and detect edges. The filter2D, manual, and separable implementations produced very similar edge maps, showing that all three methods correctly perform Sobel filtering. The separable method is also computationally more efficient because it uses two 1-D filters instead of one 2-D kernel.

8. Write a program to zoom images by a given factor $s \in (0, 10]$. You must use a function to zoom the image, which can handle

- (a) nearest-neighbor, and
- (b) bilinear interpolation.

I have included four images, two large originals, and there zoomed-out versions. Test your algorithm by computing the normalized sum of squared difference (SSD) when you scale-up the given small images by a factor of 4 by comparing with the original images.

```
def zoom_nearest(img,s):
    h,w=img.shape[:2]
    new_h=int(round(h*s))
    new_w=int(round(w*s))

    result=np.zeros((new_h,new_w,img.shape[2]),dtype=img.dtype)

    for y in range(new_h):
        src_y=min(int(round(y/s)),h-1)

        for x in range(new_w):
            src_x=min(int(round(x/s)),w-1)
```

```

        result[y,x]=img[src_y,src_x]

    return result

def zoom_bilinear(img,s):
    h,w=img.shape[:2]
    new_h=int(round(h*s))
    new_w=int(round(w*s))

    result=np.zeros((new_h,new_w,img.shape[2]),dtype=np.float32)

    for y in range(new_h):
        src_y=y/s
        y0=int(np.floor(src_y))
        y1=min(y0+1,h-1)
        dy=src_y-y0
        y0=min(y0,h-1)

        for x in range(new_w):
            src_x=x/s
            x0=int(np.floor(src_x))
            x1=min(x0+1,w-1)
            dx=src_x-x0
            x0=min(x0,w-1)

            top=(1-dx)*img[y0,x0]+dx*img[y0,x1]
            bottom=(1-dx)*img[y1,x0]+dx*img[y1,x1]

            result[y,x]=(1-dy)*top+dy*bottom

    return np.clip(result,0,255).astype(np.uint8)

def normalized_ssd(img1,img2):
    img1=img1.astype(np.float64)
    img2=img2.astype(np.float64)

    return np.mean((img1-img2)**2)/(255.0**2)

def compare_images(small_img,original_img,scale_factor=4):
    nearest=zoom_nearest(small_img,scale_factor)
    bilinear=zoom_bilinear(small_img,scale_factor)

    nearest=nearest[:original_img.shape[0],:original_img.shape[1]]
    bilinear=bilinear[:original_img.shape[0],:original_img.shape[1]]

    ssd_nearest=normalized_ssd(original_img,nearest)
    ssd_bilinear=normalized_ssd(original_img,bilinear)

    print("Nearest-neighbor NSSD:",ssd_nearest)
    print("Bilinear NSSD:",ssd_bilinear)

    images=[original_img,nearest,bilinear]

    titles=[
        "Original",
        f"Nearest Neighbor\nNSSD={ssd_nearest:.6f}",
        f"Bilinear\nNSSD={ssd_bilinear:.6f}"
    ]

    plt.figure(figsize=(15,5))

    for i in range(3):
        plt.subplot(1,3,i+1)
        plt.imshow(cv.cvtColor(images[i],cv.COLOR_BGR2RGB))
        plt.title(titles[i])
        plt.axis("off")

    plt.tight_layout()
    plt.show()

im01_small_img=cv.imread("./images/im01small.png")
im01_img=cv.imread("./images/im01.png")

im02_small_img=cv.imread("./images/im02small.png")
im02_img=cv.imread("./images/im02.png")

```

```

im03_small_img=cv.imread("./images/im03small.png")
im03_img=cv.imread("./images/im03.png")

im04_very_small_img=cv.imread("./images/im04verysmall.jpg")
im04_small_img=cv.imread("./images/im04small.jpg")
im04_img=cv.imread("./images/im04.jpg")

compare_images(im01_small_img,im01_img,4)
compare_images(im02_small_img,im02_img,4)
compare_images(im03_small_img,im03_img,4)
compare_images(im04_very_small_img,im04_small_img,4)

```

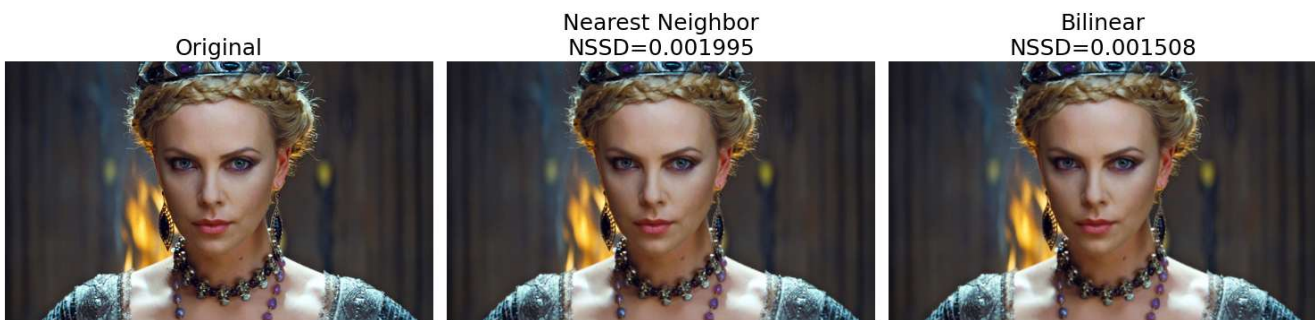
Nearest-neighbor NSSD: 0.003926116006446555
Bilinear NSSD: 0.0030795100372917666



Nearest-neighbor NSSD: 0.0009939158527881014
Bilinear NSSD: 0.0007529514111381662



Nearest-neighbor NSSD: 0.001994833762085905
Bilinear NSSD: 0.0015080711401632813



Nearest-neighbor NSSD: 0.006303533897219019
Bilinear NSSD: 0.005063400065908717



Discussion: Nearest-neighbor interpolation produces sharper but more blocky enlarged images because each output pixel is copied from the nearest source pixel. Bilinear interpolation uses surrounding pixels and therefore gives smoother results with less pixelation. The lower normalized SSD values obtained using bilinear interpolation show that it gives a closer

approximation to the original high-resolution images.

9. Fig. 8² shows a flower image with both the foreground and background are in focus.

- Use grabCut to segment the image. Show the final segmentation mask, foreground image, and background image.
- Produce an enhanced image with a substantially blurred background. Display the original image alongside the enhanced image.
- Why is the background just beyond the edge of the flower quite dark in the enhanced image?



Figure 8: Image enhancing.

```
img=cv.imread("./images/daisy.jpg")
h,w=img.shape[:2]

mask_gc=np.zeros(img.shape[:2],np.uint8)
bg_model=np.zeros((1,65),np.float64)
fg_model=np.zeros((1,65),np.float64)

rect=(int(0.08*w),int(0.10*h),int(0.84*w),int(0.65*h))

cv.grabCut(img,mask_gc,rect,bg_model,fg_model,5,cv.GC_INIT_WITH_RECT)

final_mask=np.where((mask_gc==cv.GC_FGD)|((mask_gc==cv.GC_PR_FGD),1,0).astype(np.uint8))

foreground=img*final_mask[:, :,None]
background=img*(1-final_mask[:, :,None])

blurred=cv.GaussianBlur(img,(51,51),0)

enhanced=foreground+blurred*(1-final_mask[:, :,None])

images=[img,final_mask,foreground,background,enhanced]
titles=["Original","Final Segmentation Mask","Foreground","Background","Enhanced"]

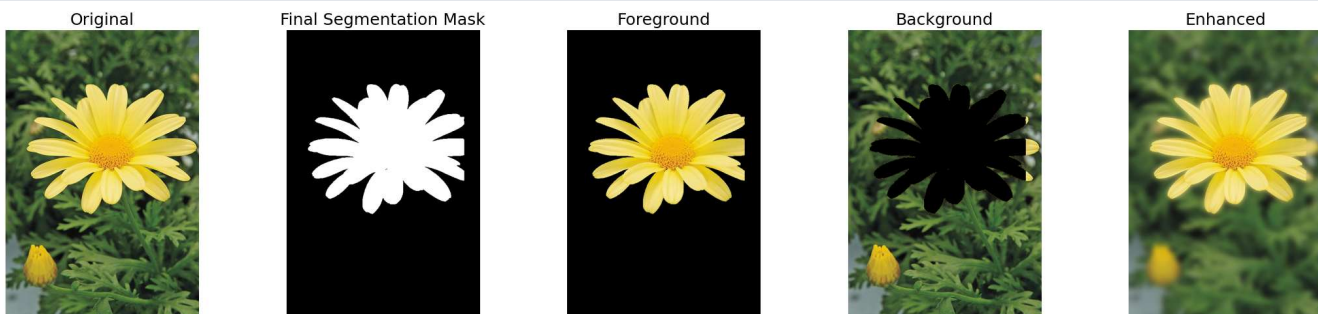
plt.figure(figsize=(22,5))

for i in range(5):
    plt.subplot(1,5,i+1)

    if i==1:
        plt.imshow(images[i],cmap="gray")
    else:
        plt.imshow(cv.cvtColor(images[i],cv.COLOR_BGR2RGB))

    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```



Discussion: GrabCut was used to separate the flower from the background, after which the background was heavily blurred while keeping the flower sharp. This creates a depth-of-field effect and draws attention to the foreground. The dark halo near the flower boundary occurs because dark pixels near the segmented edge influence the Gaussian averaging during background blurring.

10. Fig. 9 shows an image of Lake Minaret. The requirement is to smoothen the image while keeping the edges sharp.

- Use OpenCV's `bilateralFilter` to smoothen the image with a bilateral filter with σ_s and σ_r of your choice. Display the original, Gaussian blurred image with a similar kernel, and bilateral filtered images side-by-side.
- Implement your own bilateral filter and compare the results with OpenCV's `bilateralFilter`. Display the original, Gaussian blurred image with a similar kernel, and bilateral filtered images side-by-side. Use a suitable quantitative measure to compare the results.



Figure 9: Image bilateral filtering.

```
sigma_s=5
sigma_r=50
radius=4

d=2*radius+1

img=cv.imread("./images/fig9.jpeg")

gaussian=cv.GaussianBlur(img,(d,d),sigmaX=sigma_s)
bilateral_cv=cv.bilateralFilter(img,d,d,sigmaColor=sigma_r,sigmaSpace=sigma_s)

plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(cv.cvtColor(img,cv.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(cv.cvtColor(gaussian,cv.COLOR_BGR2RGB))
plt.title("Gaussian Blur")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(cv.cvtColor(bilateral_cv,cv.COLOR_BGR2RGB))
plt.title("Bilateral Filter")
plt.axis("off")

plt.tight_layout()
plt.show()

def my_bilateral_filter(img,radius=4,sigma_s=5,sigma_r=50):
    img=img.astype(np.float32)
    padded=cv.copyMakeBorder(img,radius,radius,radius,radius,cv.BORDER_REFLECT_101)
    output=np.zeros_like(img)

    yy,xx=np.mgrid[-radius:radius+1,-radius:radius+1]
    spatial_weight=np.exp(-(xx**2+yy**2)/(2*sigma_s**2))

    h,w=img.shape[:2]

    for y in range(h):
        for x in range(w):
            neighbourhood=padded[y:y+2*radius+1,x:x+2*radius+1]
            center=padded[y+radius,x+radius]

            color_difference=neighbourhood-center
            distance_squared=np.sum(color_difference**2,axis=2)

            range_weight=np.exp(-distance_squared/(2*sigma_r**2))

            weights=spatial_weight*range_weight

            weighted_pixels=neighbourhood*weights[:, :, None]

            output[y,x]=np.sum(weighted_pixels,axis=(0,1))/np.sum(weights)

    return np.clip(output,0,255).astype(np.uint8)

bilateral_own=my_bilateral_filter(img,radius,sigma_s,sigma_r)

plt.figure(figsize=(15,5))
```

```

plt.subplot(1,3,1)
plt.imshow(cv.cvtColor(img,cv.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(cv.cvtColor(bilateral_cv,cv.COLOR_BGR2RGB))
plt.title("OpenCV Bilateral")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(cv.cvtColor(bilateral_own,cv.COLOR_BGR2RGB))
plt.title("Own Bilateral")
plt.axis("off")

plt.tight_layout()
plt.show()

def mse(img1,img2):
    img1=img1.astype(np.float64)
    img2=img2.astype(np.float64)
    return np.mean((img1-img2)**2)

def psnr(img1,img2):
    error=mse(img1,img2)
    return 10*np.log10(255**2/error)

mse_value=mse(bilateral_cv,bilateral_own)
psnr_value=psnr(bilateral_cv,bilateral_own)

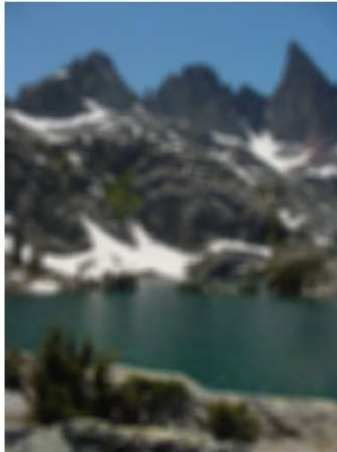
print("MSE =",mse_value)
print("PSNR =",psnr_value,"dB")

```

Original



Gaussian Blur



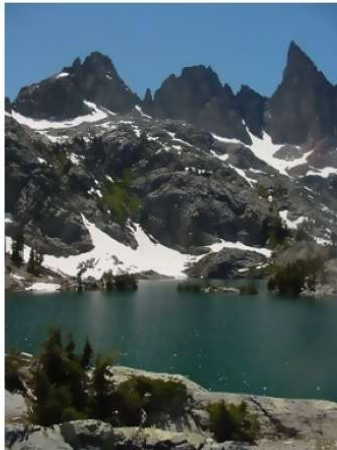
Bilateral Filter



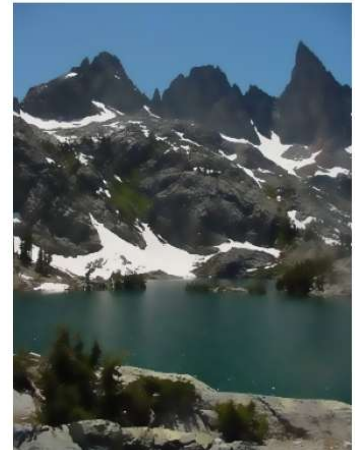
Original



OpenCV Bilateral



Own Bilateral



MSE = 17.73856571252693
 PSNR = 35.64161859708019 dB

Discussion: Gaussian filtering smooths the image but also blurs important edges because it only considers spatial distance. Bilateral filtering considers both spatial distance and intensity differences, allowing it to smooth uniform regions while preserving edges. The custom bilateral filter produced results close to OpenCV's implementation, as indicated by the low MSE and relatively high PSNR.